



University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

MoveLink

Visualisierung von Echtzeit-Bewegungsdaten

Projektbericht

Software-Architekturen Labor
Sommersemester 2026

Betreuer: Prof. Dr. rer. nat. Sinz

Team: Erlind Sejdiu
Luca Schöneberg
Devin Uyan

Abgabe: Ende Sommersemester 2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung	1
2	System Kontext, Use Cases und Anforderungen	2
2.1	System Kontext	2
3	Use Cases	3
4	Systemarchitektur	5
	Literaturverzeichnis	5

1 Einleitung

1.1 Motivation

Die Qualität der Bewegungsausführung ist im Krafttraining oder in der Physiotherapie ein wichtiger Faktor für den Trainingserfolg. Während professionelle Athleten Zugang zu Coaches und Physiotherapeuten haben, trainieren Otto-normal-Sportler ohne objektives Feedback. Fehler in der Technik, beispielsweise ein zu stark gerundeter Rücken bei der Kniebeuge, werden dadurch oft über lange Zeit nicht korrigiert und können zu Beschwerden führen.

Zwar können Bewegungsabläufe durch moderne Hardware-Sensoren erfasst werden, jedoch liegen diese Informationen oft nur als unstrukturierte Rohdaten vor, die für den Endanwender kaum interpretierbar sind. Daraus ergibt sich die Herausforderung, die anfallenden Datenströme aus verschiedenen Quellen zentral zusammenzuführen und visuell nutzergerecht aufzubereiten.

1.2 Zielsetzung

Im Rahmen dieses Laborprojekts wird die mobile Applikation **MoveLink** konzipiert und prototypisch entwickelt. Ziel der Anwendung ist es, als zentrale Schnittstelle zu fungieren, die Trainingsdaten von unterschiedlichen Hardware-Sensoren erfasst, speichert und in einer einheitlichen Benutzeroberfläche verständlich darstellt.

2 System Kontext, Use Cases und Anforderungen

2.1 System Kontext

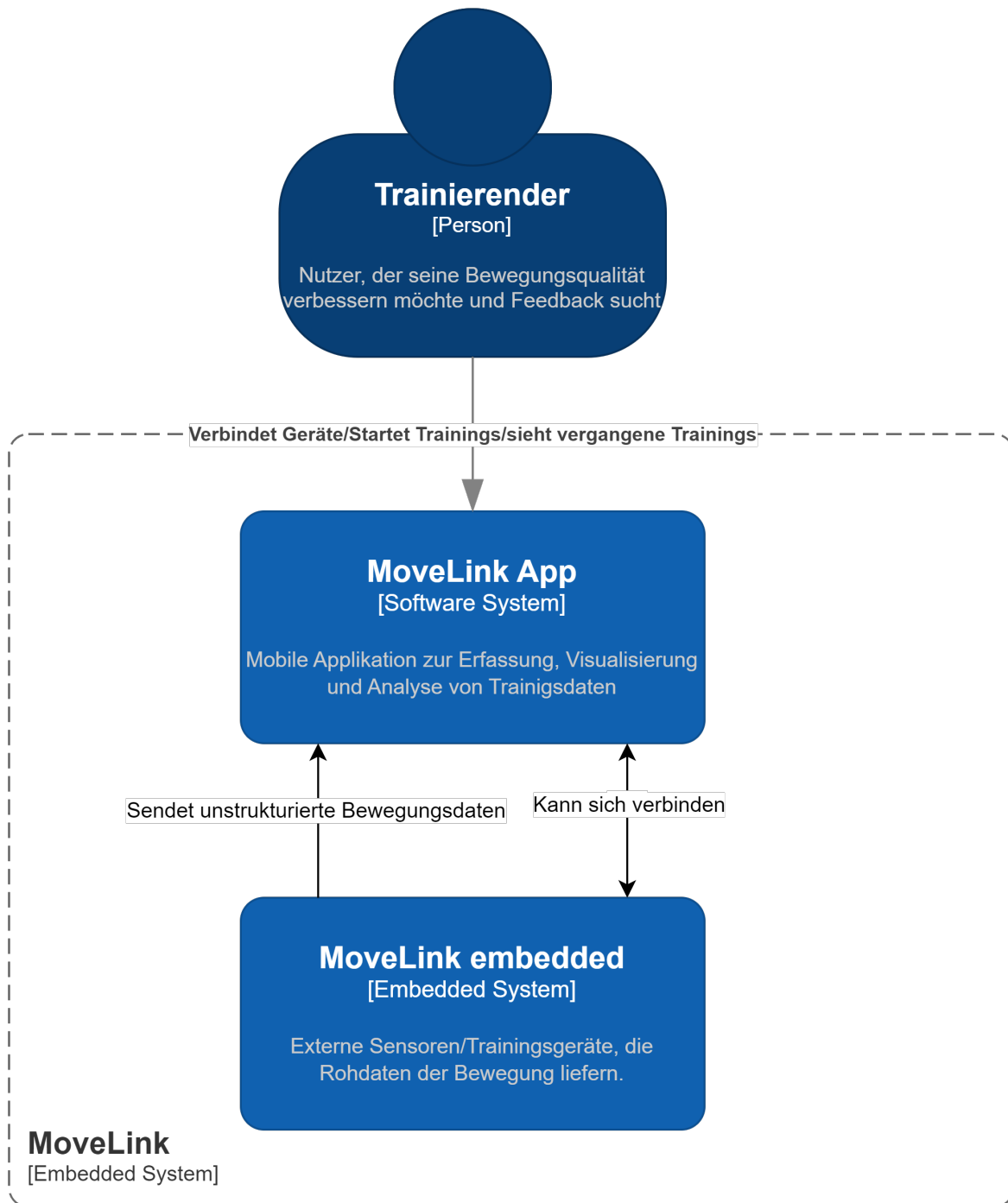


Abbildung 1: System Kontext MoveLink

3 Use Cases

Bevor eine Architektur Erläuterung folgt, sollen zunächst die folgenden Use Cases definiert werden. Diese dienen dem Verständnis der Anforderungen an das System. Jeder Use Case wird durch eine kurze Beschreibung und nachfolgend durch eine Auflistung der Schritte, die zur Erfüllung des Use Cases notwendig sind, definiert.

UC-1: Trainingsgerät verbinden

Akteur: Trainierender *Vorbedingung:* Trainingsgerät eingeschaltet

Beschreibung: Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden können, um Trainingsdaten erfassen zu können.

1. Eingabe: Ich als trainierender öffne die App, Ausgabe: Ich sehe eine Möglichkeit/Reiter Hardwaregeräte anzuzeigen.
2. Eingabe: Ich als trainierender klicke auf den Reiter Hardwaregeräte, Ausgabe: Ich sehe eine Liste der verfügbaren Hardwaregeräte und mit welchen Hardwaregeräten ich verbunden bin.
3. Eingabe: Ich als trainierender klicke auf ein Hardwaregerät in der Liste auf verbinden, Ausgabe: Ich sehe eine Detailansicht des ausgewählten Hardwaregeräts und dass dies mit der App verbunden ist.

UC-2: Echtzeit-Training überwachen

Akteur: Trainierender *Vorbedingung:* Trainingsgerät verbunden

Beschreibung: Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.

1. Eingabe: Ich als trainierender öffne die App, Ausgabe: Ich sehe eine Möglichkeit/Reiter mein Training zu starten.
2. Eingabe: Ich als trainierender klicke auf den Reiter "Training starten", Ausgabe: Ich sehe eine Detailansicht des ausgewählten Trainings und die Möglichkeit mein Training zu starten.
3. Eingabe: Ich als trainierender drücke den Start Button in der Detailansicht des Trainings, Ausgabe: Die App demonstriert mir Bewegungen, welche nachzumachen sind.
4. Eingabe: Ich als trainierender mache die Bewegung nach, Ausgabe: Die App visualisiert die Bewegung in Echtzeit und vergleicht diese mit der Demonstration. Gibt positives Feedback, wenn die Bewegung korrekt ausgeführt wurde.

UC-3: Trainingsdaten einsehen

Akteur: Trainierender *Vorbedingung:* Vergangene Trainingseinheit vorhanden

Beschreibung: Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.

1. Eingabe: Ich als trainierender öffne die App, Ausgabe: Ich sehe eine Möglichkeit/Reiter vergangene Trainingseinheiten anzuzeigen.
2. Eingabe: Ich als trainierender klicke auf den Reiter "Trainingseinheiten", Ausgabe: Ich sehe eine Liste der vergangenen Trainingseinheiten an.
3. Eingabe: Ich wähle eine Trainingseinheit aus der Liste aus, Ausgabe: Die App zeigt die historischen Bewegungsdaten grafisch an.

Funktionale Anforderungen

- **FA1:** Das System muss einen Reiter oder eine Navigationsmöglichkeit für Hardwaregeräte / Trainings / vergangene Trainingseinheiten anzeigen. (aus UC-1, UC-2, UC-3)
- **FA2:** Das System muss mir eine Liste von verfügbaren Hardwaregeräte und mit welchen Hardwaregeräten ich verbunden bin anzeigen. (aus UC-1)
- **FA3:** Das System muss einen verbindungsaufbau mit dem Hardwaregerät herstellen können. (aus UC-1)
- **FA4:** Das System muss eine Detailansicht für ein ausgewähltes Training anzeigen. (aus UC-2)
- **FA5:** Das System muss Bewegungsdatenströme empfangen und verarbeiten können. (aus UC-2)
- **FA6:** Das System muss die empfangenen Bewegungen in Echtzeit visualisieren. (aus UC-2)
- **FA7:** Das System muss historische Bewegungsdaten grafisch anzeigen können. (aus UC-3)

Nicht-funktionale Anforderungen – Muss-Kriterien

- **NF1 – Latenz:** Die E2E-Latenz vom Sensor bis zur Darstellung muss $\leq 100\text{ ms}$ sein.
- **NF2 – Zuverlässigkeit:** Bei einem Verbindungsabbruch muss die App den Nutzer benachrichtigen und automatisch Reconnect-Versuche starten.
- **NF3 – Usability:** Das Pairing darf maximal zwei Nutzerinteraktionen erfordern.

Rahmenbedingungen

- **R1:** Die Applikationen muss auf android devices laufen.

4 Systemarchitektur

Literaturverzeichnis

Literatur

- [1] Bluetooth SIG, *Bluetooth Core Specification 5.4*, 2023.
<https://www.bluetooth.com/specifications/specs/core54-html/>
- [2] Seeed Studio, *XIAO nRF52840 (Sense) Wiki*, 2023.
https://wiki.seeedstudio.com/XIAO_BLE/
- [3] STMicroelectronics, *LSM6DS3TR-C Datasheet*, 2019.
<https://www.st.com/en/mems-and-sensors/lsm6ds3tr-c.html>
- [4] I. Fette, A. Melnikov, *RFC 6455: The WebSocket Protocol*, IETF, 2011.
<https://datatracker.ietf.org/doc/html/rfc6455>
- [5] Docker Inc., *Docker Compose Documentation*, 2024.
<https://docs.docker.com/compose/>